UNIVERSIDADE DA BEIRA INTERIOR · ENGENHARIA DE SOFTWARE 2026/27

Feature Request & Product Roadmap Management System

O produto, quem o usa e o que lhe vamos pedir: personas, cenários e user stories.

Equipa

Felipe Fogaça · João Vitor · Marcus Fernandes · Rui Machado · Tiago Pereira

Sprints 1 e 2 · versão de trabalho de 17/10/2026

1. O que este documento é

> Este é o documento de base do produto. Junta o problema, os papéis que o enunciado nomeia, as sete áreas de funcionalidade, as personas, os cenários, as user stories com critérios de aceitação, os requisitos não funcionais e as interfaces candidatas.
>
> **Ainda é uma proposta.** O enunciado não traz personas escritas. As cinco personas, os cenários, as histórias e as prioridades partiram do texto do enunciado (`SE_Project.pdf` , secção *Project Proposal*) e foram escritos com apoio de IA, como diz o registo de uso de IA. As pessoas são inventadas; só os papéis e as funcionalidades vêm do enunciado. O levantamento de requisitos com o Character.ai, que o enunciado pede, ainda não foi feito e vai confirmar, corrigir ou cortar o que está aqui.

2. O produto

Nome. Development of a Feature Request & Product Roadmap Management System: um sistema para gerir pedidos de funcionalidades e o roadmap de um produto.

O problema. Um produto de software muda com o feedback dos clientes, com o que os stakeholders pedem, com o mercado e com a tecnologia. Muita gente ainda junta esses pedidos em folhas de cálculo, emails, trackers genéricos ou chats. O resultado são pedidos repetidos, prioridades que não batem certo, pouca ligação entre o que se pediu e o que se fez, e pouca visibilidade de como o produto evolui.

O que vamos construir. Um sistema com back-end e front-end em que gestores de produto, clientes, programadores e gestores de projeto trabalham no mesmo sítio, do pedido de uma funcionalidade até à release.

O que o enunciado exige à volta disto

- Back-end e front-end (recomenda Django, React ou Node.js).
- Pelo menos 8 interfaces para além do login, do registo e do CRUD básico.
- Arquitetura justificada, com wireframes e mockups.
- Requisitos levantados com o Character.ai.
- Pelo menos uma ferramenta de IA para programar.
- Jira para organizar tudo, com acesso dos docentes.
- Um Product Owner e um Team Captain (Scrum Master) por sprint, em rotação.

Fora do âmbito, por agora. Integrar ferramentas externas (por exemplo importar pedidos do email ou de um tracker). O enunciado permite, mas não pede; fica para depois do núcleo estar feito.

3. Quem usa o sistema

Estes são os papéis que o enunciado nomeia. Não são personas: as personas da secção 5 dão rosto a estes papéis.

Gestores de produto

Acompanham a evolução do produto.

Clientes

A origem de parte dos pedidos.

Programadores

Recebem trabalho e implementam.

Gestores de projeto

Acompanham o andamento.

Product Owners

O enunciado diz que a priorização manual é feita por eles.

Utilizadores que votam

O enunciado pede votação e não diz quem vota. Assumimos que são os clientes e os utilizadores do produto (a confirmar).

4. As sete áreas de funcionalidade

São as áreas do enunciado, na ordem em que ele as escreve. Essa ordem não é uma prioridade. Cada uma é um epic no Jira.

#	Área	O que cobre	Jira
1	**Feature Management**	Submeter pedidos, categorizar e etiquetar, editar com versões, pesquisar e filtrar.	EDS-48
2	**Prioritization**	Prioridade manual pelo Product Owner, votação, sugestão assistida por IA e estimativa de impacto e esforço.	EDS-49
3	**Product Backlog Management**	Criar o product backlog, definir o sprint backlog, estados das funcionalidades e atribuição a equipas.	EDS-50
4	**Product Roadmap**	Roadmap visual, planeamento de releases, milestones e dependências entre funcionalidades.	EDS-51
5	**Requirements Engineering**	Gerar user stories a partir dos pedidos, rastrear pedido, história e implementação, gerir critérios de aceitação.	EDS-52
6	**Collaboration**	Comentários e discussão, notificações, menções e espaços de trabalho por equipa.	EDS-53
7	**Dashboards and Analytics**	Pedidos submetidos, aprovados e rejeitados, funcionalidades por release, lead time, cycle time, carga das equipas e progresso do roadmap.	EDS-54

5. Personas

Cinco proto-personas, uma por papel principal. Os nomes e as histórias são inventados para ajudar a decidir. O formato segue o das aulas: quem é, o que quer, o que a incomoda.

ML

Marta Lopes

34 anos, product manager numa empresa de software B2B

Gere um produto usado por cerca de 40 empresas clientes. Recebe pedidos por email, por chat e em reuniões, e guarda-os numa folha de cálculo que só ela percebe. Tem de apresentar o plano do produto à direção de três em três meses.

O QUE QUER

Ver todos os pedidos num só sítio, perceber quais se repetem, decidir com dados o que entra em cada release e mostrar um roadmap que a direção e os clientes entendam.

O QUE A INCOMODA HOJE

Pedidos duplicados, prioridades decididas por quem insiste mais, e não conseguir explicar porque é que uma funcionalidade ficou de fora.

NO SISTEMA

Utilizadora principal do roadmap, das releases, das dependências e do dashboard. Revê a priorização.

Histórias principais: US-14

RA

Ricardo Almeida

41 anos, Product Owner de uma equipa Scrum

Trabalha com uma equipa de seis programadores em sprints de duas semanas. É ele que ordena o product backlog e aceita ou recusa o que fica pronto.

O QUE QUER

Transformar pedidos soltos em histórias claras, com critérios de aceitação, ordenar o backlog com impacto e esforço à vista e montar o sprint backlog sem folhas de cálculo.

O QUE A INCOMODA HOJE

Pedidos mal descritos que obrigam a perguntar tudo de novo, estimativas que ninguém regista e perder o fio entre o que se pediu e o que a equipa entregou.

NO SISTEMA

Utilizador principal da priorização, do backlog, das user stories e dos critérios de aceitação.

Histórias principais: US-06, US-08, US-11, US-18

SF

Sofia Ferreira

29 anos, responsável de operações numa empresa cliente

Usa o produto todos os dias e vai notando o que falta. Hoje manda os pedidos por email ao seu contacto na empresa fornecedora e raramente sabe o que lhes aconteceu.

O QUE QUER

Submeter um pedido em poucos minutos, ver se alguém já pediu o mesmo, votar nos pedidos que a afetam e saber em que estado está o que pediu.

O QUE A INCOMODA HOJE

Não ter resposta, não saber se o pedido foi lido e ter de repetir o mesmo pedido de cada vez que muda o interlocutor.

NO SISTEMA

Cliente que submete, vota e acompanha. É também a nossa aproximação ao *utilizador que vota*.

Histórias principais: US-01, US-07, US-21

DM

Diogo Martins

26 anos, programador full-stack

Está há dois anos na equipa. Recebe tarefas pelo Jira e por mensagens. Gosta de trabalhar com requisitos claros e de saber o motivo do que está a construir.

O QUE QUER

Ver a funcionalidade que lhe atribuíram com a história, os critérios de aceitação e as dependências, comentar com quem pediu, e ligar o que fez ao pedido original.

O QUE A INCOMODA HOJE

Pedidos vagos, mudanças de âmbito sem aviso e não saber que outra funcionalidade tem de estar pronta antes da sua.

NO SISTEMA

Utilizador da atribuição de funcionalidades, do estado, das menções e da rastreabilidade.

Histórias principais: US-12

HC

Helena Costa

45 anos, gestora de projeto

Acompanha duas equipas e os prazos de cada release. Quer perceber a carga de cada equipa e se o plano se mantém, sem ter de interromper as pessoas para perguntar.

O QUE QUER

Ver o progresso do roadmap, a carga das equipas e o tempo que as funcionalidades levam do pedido à entrega, para ajustar prazos e avisar a direção a tempo.

O QUE A INCOMODA HOJE

Relatórios feitos à mão que já nascem desatualizados e descobrir os atrasos tarde demais.

NO SISTEMA

Utilizadora principal dos dashboards, das métricas e dos milestones.

Histórias principais: US-27

6. Cenários

Cada cenário é o caminho de uma persona a tentar um objetivo. Inclui o que o sistema responde e o que acontece quando corre mal.

C1. Sofia submete e acompanha um pedido

A Sofia repara que os relatórios do produto não se exportam para Excel. Entra no sistema, escreve o título e a descrição do pedido e escolhe a categoria *Relatórios*. Enquanto escreve, o sistema mostra dois pedidos parecidos. Um deles é o que ela queria: vota nele em vez de submeter outro. Dias depois recebe uma notificação a dizer que o pedido passou a *Em análise*.

Quando corre mal. Se o pedido que ela submete for mesmo duplicado e ela ignorar o aviso, o pedido fica criado mas o Ricardo pode juntá-lo ao original. A Sofia é avisada e os votos passam para o pedido principal.

C2. Ricardo prioriza o backlog

No início da semana o Ricardo abre a lista de pedidos novos. Filtra os que têm mais votos, regista para cada um o impacto e o esforço e vê a sugestão de prioridade da IA. Concorda com duas sugestões e discorda de uma: mantém a ordem dele e escreve a razão. Passa os pedidos escolhidos para o product backlog.

Quando corre mal. Se a IA não estiver disponível, o sistema avisa e o Ricardo prioriza à mão. A IA só sugere, nunca decide. Se a estimativa de esforço ainda não existir, o pedido não entra no sprint backlog.

C3. Marta monta uma release no roadmap

A Marta abre o roadmap visual, cria a release 2.0 com um milestone para o fim de novembro e arrasta para lá as funcionalidades aprovadas. Ao arrastar *Exportar relatórios*, o sistema avisa que depende de *Filtros avançados*, que ainda não está agendada. Ela agenda as duas.

Quando corre mal. Se criar uma dependência circular (A depende de B e B depende de A), o sistema recusa e explica porquê.

C4. Ricardo gera histórias a partir de um pedido

O pedido *Exportar relatórios* foi aprovado. O Ricardo pede ao sistema para gerar as user stories. Aparecem três propostas no formato *como… quero… para…*. Edita uma, rejeita outra e acrescenta critérios de aceitação às que ficam. O sistema liga as histórias ao pedido original.

Quando corre mal. Se o pedido tiver pouca informação, o sistema gera menos histórias e diz o que falta. As histórias geradas nunca entram no backlog sem o Ricardo as aceitar.

C5. Diogo trabalha numa funcionalidade

O Diogo vê no seu espaço de equipa a funcionalidade que lhe atribuíram, com as histórias e os critérios. Muda o estado para *Em desenvolvimento*, escreve um comentário com uma pergunta e menciona a Sofia para esclarecer um detalhe. No fim liga o seu *pull request* à funcionalidade e passa-a a *Em revisão*.

Quando corre mal. Se tentar passar a *Concluída* sem pull request ligado, o sistema avisa que falta a ligação à implementação.

C6. Helena verifica o andamento

Antes da reunião com a direção, a Helena abre o dashboard. Vê o progresso do roadmap, a carga de cada equipa e o lead time e o cycle time médios. Repara que uma equipa está acima da capacidade e que a release 2.0 está atrasada. Exporta a vista e pede ao Ricardo para rever o âmbito.

Quando corre mal. Se faltarem dados (por exemplo funcionalidades sem estimativa), o dashboard mostra a métrica como incompleta e diz quantos itens faltam, em vez de mostrar um número que engana.

7. User stories principais

Aqui ficam as histórias principais, uma ou duas por área. O backlog completo está no Jira. Formato das aulas: como *persona*, quero *objetivo*, para *benefício*, com a prioridade proposta (MoSCoW) e os critérios de aceitação. A geração automática de histórias (US-18) é uma funcionalidade do produto e não se confunde com estas.

A prioridade é uma proposta. Quem decide a ordem do backlog é o Product Owner.

Feature Management

Submeter pedidos, categorizar e etiquetar, editar com versões, pesquisar e filtrar.

US-01

MUST

Como *cliente*, quero *submeter um pedido de funcionalidade com título, descrição e categoria*, para *o meu pedido ficar registado e não se perder num email*.

CRITÉRIOS DE ACEITAÇÃO

- O pedido só se grava com título e descrição preenchidos.
- Depois de gravar, o pedido aparece na lista com o estado *Novo*.
- Quem submete recebe a confirmação no ecrã.

US-04

MUST

Como *utilizador*, quero *pesquisar e filtrar pedidos por texto, estado, categoria e etiqueta*, para *encontrar depressa o que procuro*.

CRITÉRIOS DE ACEITAÇÃO

- A pesquisa por texto procura no título e na descrição.
- Os filtros combinam-se entre si.
- Quando não há resultados, o ecrã diz isso e sugere limpar os filtros.

Prioritization

Prioridade manual pelo Product Owner, votação, sugestão assistida por IA e estimativa de impacto e esforço.

US-06

MUST

Como *Product Owner*, quero *ordenar manualmente os pedidos por prioridade*, para *a equipa trabalhar primeiro no que tem mais valor*.

CRITÉRIOS DE ACEITAÇÃO

- Só o Product Owner altera a ordem.
- A alteração fica registada com data e autor.
- O Product Owner pode escrever a razão da ordem.

US-07

MUST

Como *utilizador*, quero *votar nos pedidos que me interessam*, para *mostrar o que mais precisa de ser feito*.

CRITÉRIOS DE ACEITAÇÃO

- Cada utilizador tem um voto por pedido e pode retirá-lo.
- O número de votos é visível na lista e no pedido.
- O sistema impede votar duas vezes no mesmo pedido.

US-08

SHOULD

Como *Product Owner*, quero *receber uma sugestão de prioridade assistida por IA*, para *apoiar a minha decisão com outra leitura dos dados*.

CRITÉRIOS DE ACEITAÇÃO

- A sugestão usa dados do pedido (votos, impacto, esforço).
- A sugestão é só uma recomendação: nada muda sem o Product Owner aceitar.
- Quando a IA falha ou não responde, o resto da priorização continua a funcionar.

Product Backlog Management

Criar o product backlog, definir o sprint backlog, estados das funcionalidades e atribuição a equipas.

US-11

MUST

Como *Product Owner*, quero *definir o sprint backlog escolhendo itens do product backlog*, para *a equipa saber o que fazer em cada sprint*.

CRITÉRIOS DE ACEITAÇÃO

- Um item só entra no sprint se tiver estimativa.
- O sprint tem datas e objetivo.
- O que não fica pronto volta ao product backlog.

US-12

MUST

Como *programador*, quero *atualizar o estado de uma funcionalidade*, para *todos verem em que ponto está*.

CRITÉRIOS DE ACEITAÇÃO

- Os estados são *Novo*, *Em análise*, *Aprovado*, *Rejeitado*, *Em desenvolvimento*, *Em revisão* e *Concluído*.
- Cada mudança de estado fica registada com data e autor.
- Os interessados são notificados quando o estado muda.

Product Roadmap

Roadmap visual, planeamento de releases, milestones e dependências entre funcionalidades.

US-14

MUST

Como *product manager*, quero *ver o roadmap numa linha do tempo visual*, para *perceber de relance o que vem e quando*.

CRITÉRIOS DE ACEITAÇÃO

- O roadmap mostra releases e funcionalidades ao longo do tempo.
- Arrastar uma funcionalidade para outra release atualiza-a.

Requirements Engineering

Gerar user stories a partir dos pedidos, rastrear pedido, história e implementação, gerir critérios de aceitação.

US-18

SHOULD

Como *Product Owner*, quero *gerar user stories a partir de um pedido aprovado*, para *poupar tempo a escrever e não perder nada do que foi pedido*.

CRITÉRIOS DE ACEITAÇÃO

- As histórias saem no formato *como… quero… para…*.
- O Product Owner edita, aceita ou rejeita cada uma.
- Cada história aceite fica ligada ao pedido de origem.

Collaboration

Comentários e discussão, notificações, menções e espaços de trabalho por equipa.

US-21

MUST

Como *utilizador*, quero *comentar e discutir um pedido*, para *esclarecer dúvidas no sítio onde está o pedido*.

CRITÉRIOS DE ACEITAÇÃO

- Os comentários mostram autor e data.
- O autor pode editar ou apagar o seu comentário.

Dashboards and Analytics

Pedidos submetidos, aprovados e rejeitados, funcionalidades por release, lead time, cycle time, carga das equipas e progresso do roadmap.

US-27

SHOULD

Como *project manager*, quero *ver o lead time e o cycle time*, para *perceber quanto tempo demora uma funcionalidade do pedido à entrega*.

CRITÉRIOS DE ACEITAÇÃO

- Lead time: da data do pedido à conclusão.
- Cycle time: do início do desenvolvimento à conclusão.
- O dashboard mostra a média e a evolução por período.

8. Requisitos não funcionais

Valores propostos, por confirmar com a equipa. O enunciado pede arquitetura justificada, modularidade e escalabilidade.

Requisito	O que significa
Usabilidade	Submeter um pedido não exige mais do que um ecrã. A interface funciona em portátil e em telemóvel.
Desempenho	A pesquisa e as listas respondem em menos de 2 segundos com alguns milhares de pedidos.
Segurança	Autenticação, perfis com permissões por papel, palavras-passe com hash e proteção contra os erros mais comuns (injeção, XSS).
Rastreabilidade e auditoria	Mudanças de estado, de prioridade e de versão ficam registadas com autor e data. O mesmo vale para as sugestões da IA.
Privacidade	Só se guardam os dados pessoais necessários e cumpre-se o RGPD.
Fiabilidade	Se a IA ou um serviço externo falhar, o resto do sistema continua a funcionar.
Manutenção	Código modular, testes automáticos e integração contínua na `main`, como o contrato diz.
Escalabilidade	Arquitetura que permita crescer em utilizadores e em equipas sem refazer tudo.
Acessibilidade	Contraste, navegação por teclado e textos alternativos.
Idioma	Interface em português, com estrutura para acrescentar inglês.

9. Interfaces candidatas e wireframes

O enunciado pede pelo menos 8 interfaces para além do login, do registo e do CRUD básico. Estas dez são as candidatas; os wireframes do Sprint 2 escolhem e desenham as principais.

1. Roadmap visual com releases e milestones

2. Quadro de priorização, com votos, impacto, esforço e sugestão da IA

3. Detalhe do pedido, com versões, comentários e menções

4. Gerador e editor de user stories com critérios de aceitação

5. Planeamento de sprint (product backlog e sprint backlog)

6. Vista de rastreabilidade (pedido, história, implementação)

7. Dashboard de métricas (pedidos, lead time, cycle time, release)

8. Carga das equipas

9. Centro de notificações

10. Espaço de trabalho de equipa

Exemplos de wireframes

Rascunhos de baixa fidelidade, só para discutir a estrutura de alguns ecrãs. Não são os wireframes finais: esses ficam no Figma e nos cartões EDS-97 a EDS-103.



1. Submeter um pedido. Enquanto se escreve o título, o sistema mostra pedidos parecidos para votar em vez de duplicar. US-01 e cenário C1.



2. Lista de pedidos com pesquisa e filtros. US-04 e US-07.

Priorização — Prioritization

#	Pedido	Votos	Impacto	Esforço	IA sugere
≡ 1	Exportar relatórios	14	Alto	M	Alta
≡ 2	Filtros avançados	7	Médio	S	Alta
≡ 3	Notificações por email	9	Médio	L	Média

Razão da ordem

Escreve porque este vem primeiro…

Aceitar sugestão | Manter a minha

3. Priorização. A ordem é arrastada à mão pelo Product Owner; a IA só sugere. US-06 e US-08, cenário C2.

Roadmap — Product Roadmap

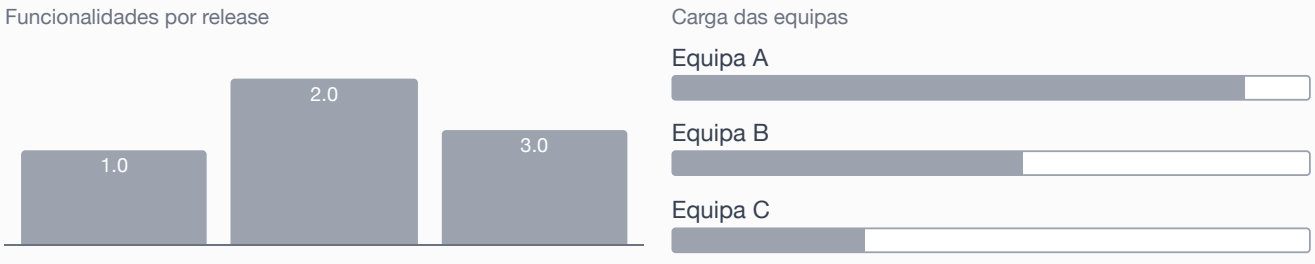


◆ milestone · arrastar uma barra muda a funcionalidade de release · o sistema avisa se uma dependência fica para depois.

4. Roadmap visual. US-14, cenário C3.

Dashboard — Dashboards and Analytics

Submetidos	Aprovados	Rejeitados	Lead time	Cycle time
128	41	17	12 d	5 d



5. Dashboard. Os números são inventados, só para mostrar onde fica cada métrica. US-27, cenário C6.

Wireframes no Figma

Cada área do enunciado tem um cartão de wireframe no Jira. O link do Figma fica no cartão e no EDS-26.

Área	Cartão	Ecrã no Figma
Feature Management	EDS-97	**link por juntar**
Prioritization	EDS-98	**link por juntar**
Product Backlog Management	EDS-99	**link por juntar**
Product Roadmap	EDS-100	**link por juntar**
Requirements Engineering	EDS-101	**link por juntar**
Collaboration	EDS-102	**link por juntar**
Dashboards and Analytics	EDS-103	**link por juntar**

10. De onde vem cada persona

Persona	Papel	Histórias principais
Marta Lopes	product manager	US-14
Ricardo Almeida	Product Owner	US-06, US-08, US-11, US-18
Sofia Ferreira	cliente	US-01, US-07, US-21
Diogo Martins	programador	US-12
Helena Costa	gestora de projeto	US-27
Todos		US-04

11. O que falta e os limites

- **Character.ai.** Falta o levantamento. Quando houver, confirmar personas, cenários e histórias.
- **Validação pela equipa.** Ninguém além de quem escreveu leu este documento.
- **Prioridades.** São uma proposta; o Product Owner decide.
- **Escalas e números.** A escala de impacto e esforço, os tempos de resposta e os estados são valores propostos.
- **Enunciado.** A página 3 é um esquema do percurso da cadeira (SWEBOK v4 e SFIA 9), não uma lista de ecrãs. As datas do enunciado não são as do grupo, que estão no contrato e no Jira.
- **Lab03.** O Smart Campus é o exercício da aula, usado só para o método. Nenhuma persona nem história dali entra aqui.
- **Terminologia.** O enunciado chama Team Captain ao Scrum Master. No contrato e na equipa continua Scrum Master.
- **Apresentação.** A apresentação oral do Sprint 1 e a do Sprint 2 fazem-se juntas (combinado da equipa, não vem do enunciado).